



Internet of Things



Chapter 3: Digital Interfaces

PR. OUASSIM KARRAKCHOU

UNIVERSITE INTERNATIONALE DE RABAT



Digital Interfaces

Advanced digital communication

We mostly saw so far sensors and actuators that are controlled using a simple digital or analog signal. The device can communicate with them simply using:

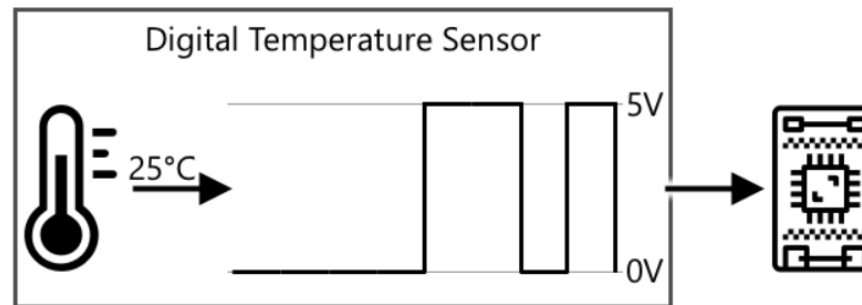
- **Digital GPIO pins:** i.e., with 2 states like the LED or the push button
- **Digital GPIO with PWM output:** useful to approximate an analog output for actuators that have an averaging effect like the LED brightness and the speed of a motor.
- **Analog input:** to measure a voltage that is related to a certain physical quantity (e.g., light or temperature value) or that corresponds to a variable user input (e.g., potentiometer).



Digital Interfaces

Advanced digital communication

Advanced digital sensors and actuators communicate using a complex **sequence of bits** that needs to be **decoded**. For example, a **temperature** sensor may directly measure the temperature and send it in a specific binary format to the IoT device:



⇒ There are different ways to transmit a sequence of bits!

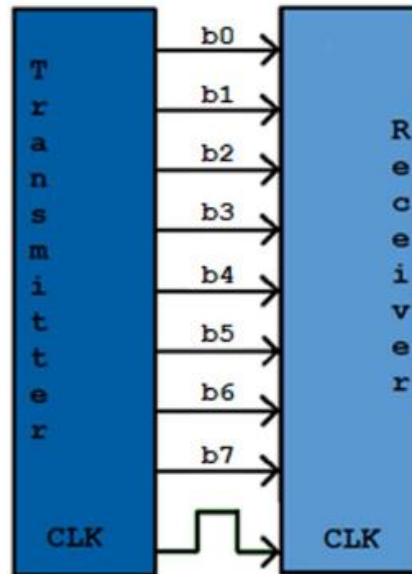
⇒ We need a communication protocol that specifies how each side needs to communicate with the other



Digital Interfaces

Simplest digital interface: the parallel interface

The simplest digital interface one can think of to send several bits at once is the **parallel interface**: we use **several pins each one transmitting a bit** in parallel. With 8 wires, we can thus send a byte:



8-bit Parallel Interface

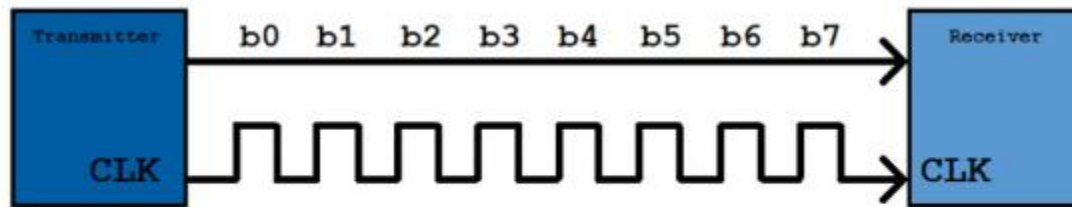
=> The parallel interface needs a lot of wires!



Digital Interfaces

A cheaper digital interface: the serial interface

An alternative to the parallel interface is to send every bit in a sequence in a single wire: it's the serial interface:



8-bit Serial Interface

- ⇒ The serial interface uses less wires, but it is also slower as it only sends a single bit per clock pulse
- ⇒ For IoT applications, we are usually more concerned about cost and power consumption rather than speed. So, serial interfaces are the most used for IoT devices.



Digital Interfaces

Serial interfaces

There are several serial interface standards that exist and that differ in:

- **How many devices** can be connected to the same wire
- Their **speed**
- Their **framing**: i.e., how data is identified in the transmitted bit stream
- Whether they are **synchronous** or **asynchronous**: i.e., if they need a clock signal
- The **error correction**
- Etc...

⇒ The most popular serial interface you know is the **USB** interface

⇒ However, USB is a complex interface with handshaking, device detection, auto speed negotiation etc. So, there are slower and simpler serial interfaces used in IoT



Digital Interfaces

The UART interface

The **UART** interface is usually what is referred to colloquially by the “**Serial**” interface in several electronic devices.

UART stands for **Universal Asynchronous Receiver/Transmitter**. The UART is **very cheap**, it allows **two** devices to communicate with each other using only **two wires** (one for each direction) plus the ground.

A **UART** may be used when:

- High speed is not required
- A cheap communication line between two devices is required

⇒ The **Serial** object you used in the **Arduino framework** allows you to access the **UART** interface of the **Arduino boards**



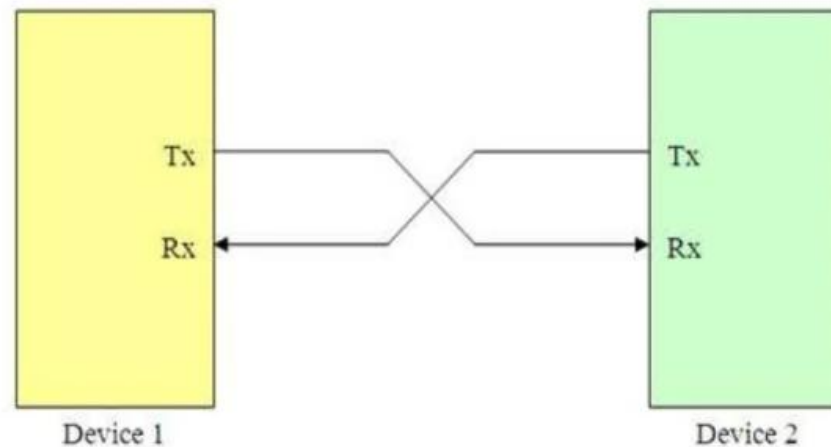
Digital Interfaces

The UART interface

Every device has two pins:

1. **RX:** the receiver pin through which it receives data
2. **TX:** the transmitter pin from which it transmits data

When two devices are connected, the TX pin of one is connected to the RX pin of the other:





Digital Interfaces

The UART interface

UART allows sending a sequence of bits between two devices. There are several standards that allow to specify which **voltage** level corresponds to a **LOW** or a **HIGH** bit. Usually, microcontrollers use **TTL-level UART** that is compatible with the voltage level of normal GPIO pins:

LOGIC	VOLTAGE
LOW	0V
HIGH	5V

However, **some devices expect a 3.3V as a HIGH** bit in the UART interface.

=> In this case, you need to use a **voltage divider** at the entrance of the **RX pin of the 3.3V** device to convert the 5V HIGH coming from the TX of the 5V device.



Digital Interfaces

Quizz

Can we control an analog actuator using an Arduino? What technique can we use to do it?



Digital Interfaces

How UART communicates?

1. The UART **transmission line** is normally held **HIGH** when no transmission is happening.
2. When the transmitter wants to transmit, it pulls the **transmission line to LOW** for one clock cycle: it the **start bit**
3. When the receiver detects the HIGH to LOW transition, it starts **reading the bits at a specific frequency**
4. A **data frame can have 5 to 8 bits** in UART and is sent with the **least-significant bit first**
5. After the data frame is sent, the transmitter can send an **optional parity bit for error correction**
6. To signal the **end of the transmission**, the transmitter **puts the line to HIGH for 1 or 2 bit times**



Digital Interfaces

How UART communicates?

The start bit allows the transmitter to signal to the receiver that it is about to send data.

The parity bit is a simple form of error detection where the number of bits in the data frame (i.e., the data bits + the parity bit) is either **even or odd**. If a bit changed during transmission, it will be detected.

The stop bit allows the transmitter to signal to the receiver that it finished sending data. Both the start and stop bit constitute a **voltage change pattern** that allows the receiver to identify data frames.

Both the receiver and the transmitter need to have the **same configuration** in terms of:

- The **number of bits in the data frame** (between 5 and 8)
- Whether a **parity bit** is used and its **type (even or odd)**
- The **size of the stop bit** (1, 1.5, or 2 bit times)

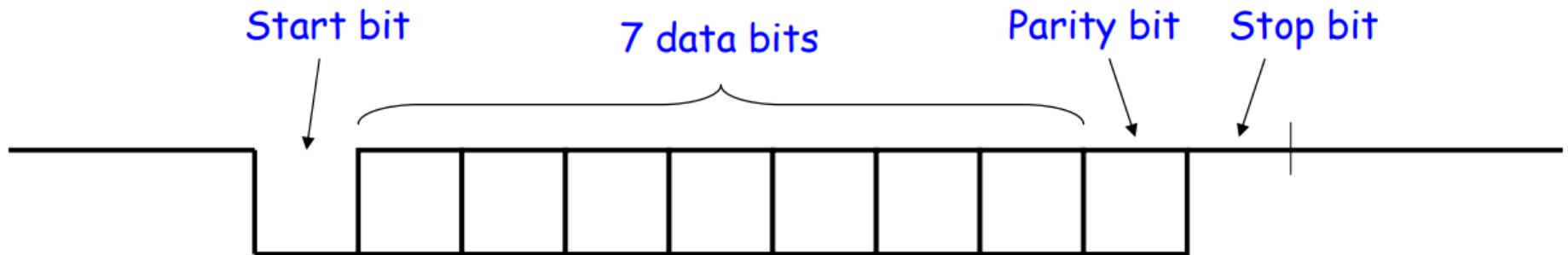
By default, UART uses 8 bit data frames, with no parity bit, and a stop bit of size 1 (referred to as 8N1).



Digital Interfaces

How UART communicates?

This is how the transmission looks like for 7 data bits, 1 parity bit, and a stop bit of size 1: we need 10 bits to transmit 7 bits

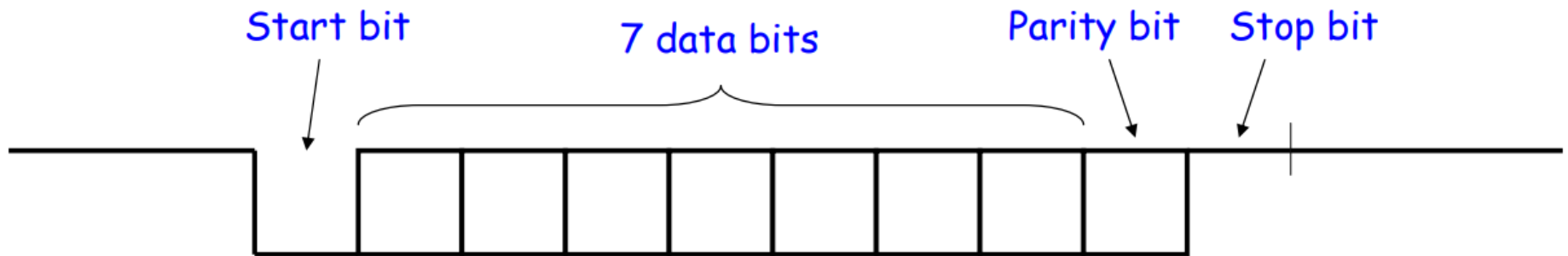




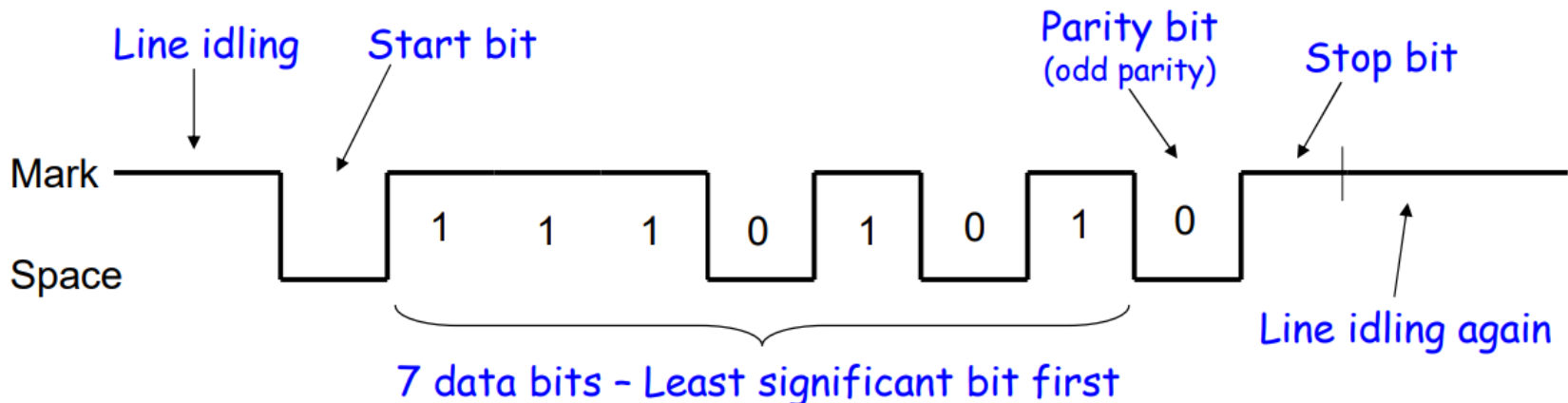
Digital Interfaces

How UART communicates?

This is how the transmission looks like for 7 data bits, 1 parity bit, and a stop bit of size 1: we need 10 bits to transmit 7 bits



Example for sending the ASCII character W (1010111):



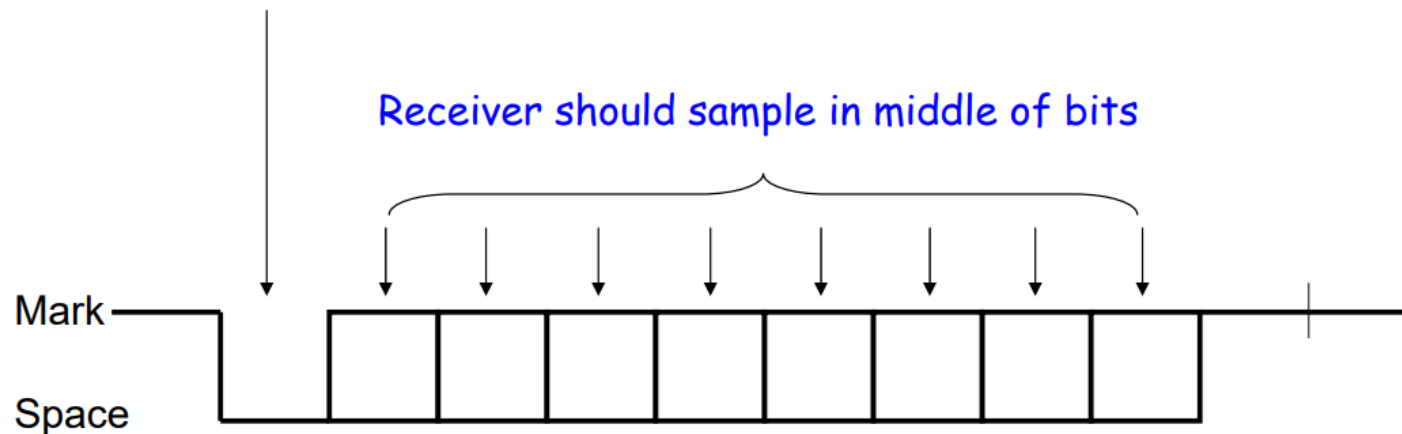


Digital Interfaces

How UART communicates

From the receiver point of view, this is what happens:

Start bit says a character is coming,
receiver resets its timers



Receiver uses a timer (counter) to time when it samples.
Transmission rate (i.e., bit width) must be known!

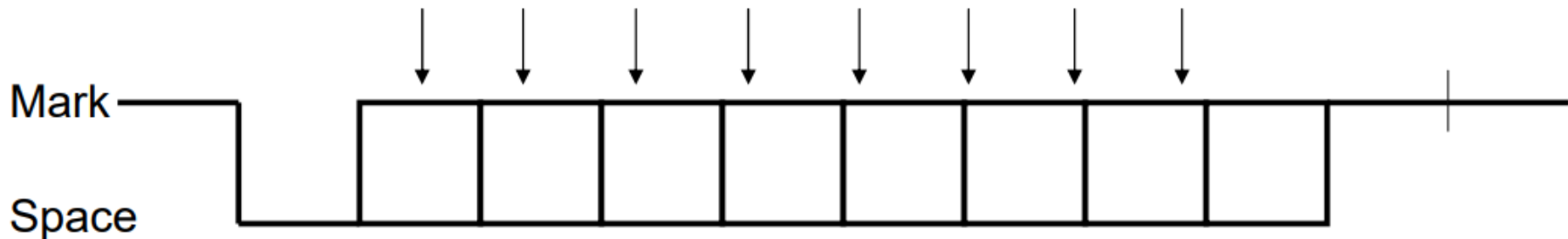


Digital Interfaces

How UART communicates

From the receiver point of view, this is what happens:

If receiver samples too quickly, see what happens...



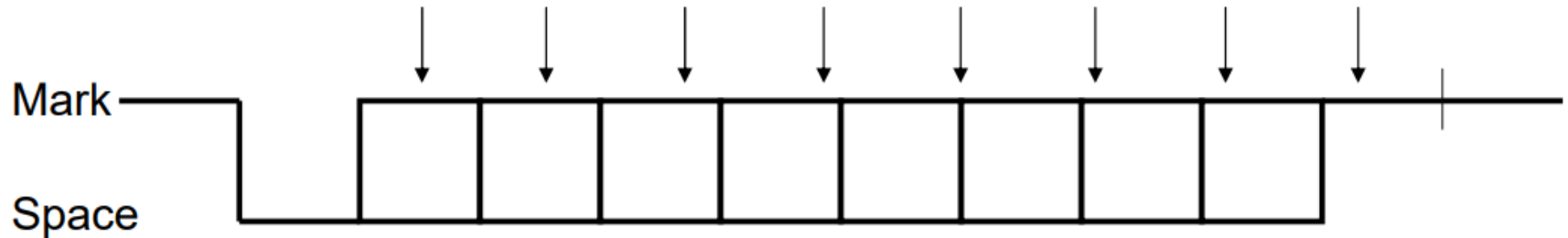


Digital Interfaces

How UART communicates

From the receiver point of view, this is what happens:

If receiver samples too slowly, see what happens...



Receiver resynchronizes on every start bit.
Only has to be accurate enough to read 9 bits.



Digital Interfaces

How UART communicates

Both the receiver and the transmitter need to use the same **Baud Rate** to sample the bit at the correct frequency.

What is the Baud rate?

- The “**symbol rate**” of the transmission system (some transmission systems can use complex modulation techniques to put more bits per symbol, for example by using multiple voltage levels)
- For a **UART**, it is the same as the **number of bits per second** (bps)
- But it is not the same as the **throughput** as in 8N1 UART, we need 10 bits to transmit a data frame of 8 bits of useful data!



Digital Interfaces

Using UART with Arduino

In **Arduino**, we can use the **Serial** object to communicate using the UART interface (through the RX and TX pins of the board, which are also connected to the USB).

To initialize the serial interface, we have to use **Serial.begin(baudRate)** to specify the correct baud rate for the transmission.

To transmit data through the Serial interface, we can use the following functions:

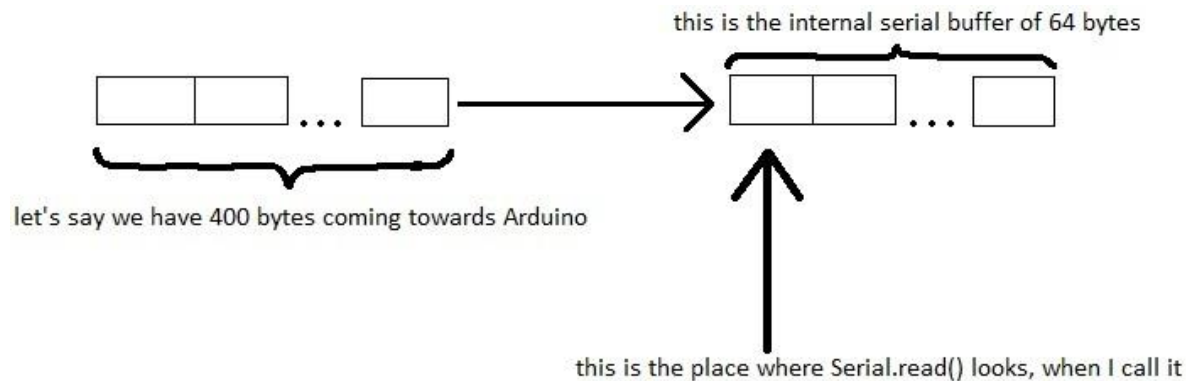
- **Serial.print(string)**: send the string through the serial interface
- **Serial.println(string)**: send the string through the serial interface followed by a new line character
- **Serial.write(buffer, length)**: sends a buffer (i.e., a C array of binary data) through the serial interface



Digital Interfaces

Using UART with Arduino

When **data is received** through the UART interface, it is automatically stored in an **internal buffer of size 64 bytes** managed by the Serial library:



You can then **read data from this buffer** in the Arduino code using the following functions:

- **`Serial.available()`**: returns the number of bytes of new data in the UART receive buffer
- **`Serial.read()`**: reads a single byte from the UART receive buffer
- **`Serial.readString()`**: reads a string of character from the UART receive buffer until a timeout occurs. The exact value of the timeout in milliseconds can be set with the **`Serial.setTimeout(millis)`** function
- **`Serial.readBytes(buffer, length)`**: reads a specified number of bytes from the UART receive buffer into the specified buffer.



Digital Interfaces

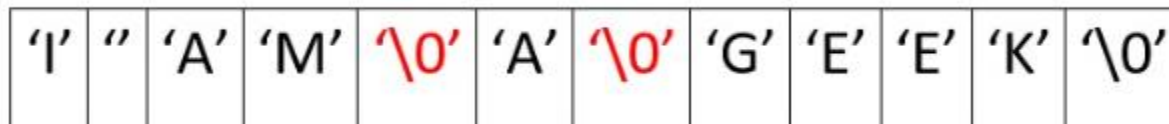
The code for receiving messages:

It is a good practice to always receive the messages **byte by byte** if some bytes are **available** in the serial interface internal receive buffer:

```
while (BTSerial.available())  
{  
    // We read the last character and add it to the receive buffer  
    char c = BTSerial.read();  
}
```

For the reception of message, we have to adopt a convention to be able to **separate between different consecutive messages**.

This is called **framing**, and it consists of selecting a **certain condition** that will determine the **start and stop of the messages** in the stream of bytes received through the serial interface:



=> When we have a stop condition with no start condition, we have a delimiter



Digital Interfaces

The code for receiving messages:

In our case, we are usually using text messages, so we adopt the convention that **new lines** separate messages:

- We have **no start condition** (messages start right away)
- Our **stop condition is the reception of a new line** (i.e., `\n`, or `\r` followed by `\n` or `\0`).

```
// Receive messages from the Bluetooth receiver
String receiveBuffer;
char lastCharacterReceived = 0;
const size_t MAX_MESSAGE_LENGTH = 1000;
```

```
while (BTSerial.available())
{
    // We read the last character and add it to the receive buffer
    char c = BTSerial.read();
    receiveBuffer += c;
    // Message end condition: receiving a \n or a \r followed by \n
    if (c == '\n' || (c == '\r' && lastCharacterReceived == '\r')
        || receiveBuffer.length() == MAX_MESSAGE_LENGTH)
    {
        // We have a new message
    }
}
```



Digital Interfaces

Using UART with Arduino

You can also create a new UART interface using other pins of the Arduino board than the RX and TX pins using the **SoftwareSerial** library (you have to include **SoftwareSerial.h**). You can then create a new software serial attached to different pins using the following declaration (outside `setup()` and `loop()`):

- **SoftwareSerial SoftSerial(NEW_RX_PIN, NEW_TX_PIN);**

⇒ SoftSerial has the same functions as the Serial object!

⇒ Careful: NEW_RX_PIN and NEW_TX_PIN, correspond to the RX and TX pins of the new serial interface **from the point of view of the Arduino board**. They need to be connected to the TX and RX pin of the device respectively.



Digital Interfaces

UART to USB converters

Inside the Arduino board, there is a special circuit that converts UART to USB. The Arduino board then appears in the computer as a **serial port** (COM port).

⇒ This is why we have access to the data sent using `Serial.print()` in the computer.

There are also standalone circuits that allow to convert from USB to UART colloquially referred to as an “FTDI chip”:

